

Relational Stock Database

Jaslyn Chan

1.0 Introduction

1.1 Purpose

This project focuses on analysing stock price and trading volume data for a group of large technology companies. By structuring and querying this data in a relational database, I used SQL queries to extract meaningful insights such as returns, volatility, and trading activity.

1.2 Dataset Assessment

I downloaded the dataset from Stooq, a public repository that gives historical market data. The site can be found easily, and downloading files does not require an account or API key. This supports reproducibility, as the same tickers and date ranges can be obtained again from the same place.

The dataset provides daily records with standard market fields and trading dates. Values are numeric and structured and the daily time series format is good for database storage and analysis. One limitation is that financial market datasets can include non-trading days like missing dates due to weekends or holidays. But this is fine and does not reduce the usefulness of the data.

1.3 Terms of use

Stooq provides publicly accessible downloads intended for informational use. The data was used for educational purposes in a university coursework project. I did not bypass any paywalls or scrape data beyond what is allowed. The dataset is not redistributed commercially.

1.4 Dataset Interest

This dataset is interesting because it captures real-world market behaviour over time and lets us compare between companies under changing market conditions. The information is very recent, with data for intervals of up to five minutes, with an approximate 15 minute delay. This supports meaningful questions about performance (returns), risk (volatility) and trading activity (volume) which are widely used in financial and business contexts.

1.5 Dataset Description

The dataset has daily stock price data (open, high, low, close prices, trading volume). Each record is one trading day for one company. Data was collected for seven publicly traded companies. Apple, Microsoft, Amazon, Google, Meta, Nvidia, and Tesla over approximately the last five years.

I cropped the data to remove older records, leaving only the last five years to ensure a consistent time window for all companies.

1.4 Queries

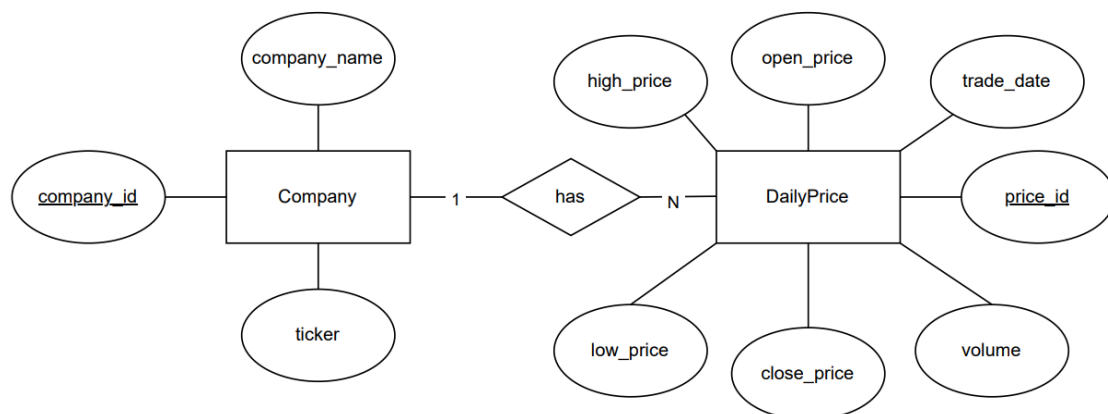
This project answers the questions:

- How many records exist for each company, and what date range is covered?
- Which company achieved the highest overall return over the dataset period?
- Which company exhibits the highest day-to-day price volatility?
- Which company has the highest average trading volume?
- How do average closing prices compare across companies?

2.0 Data Modelling & Design

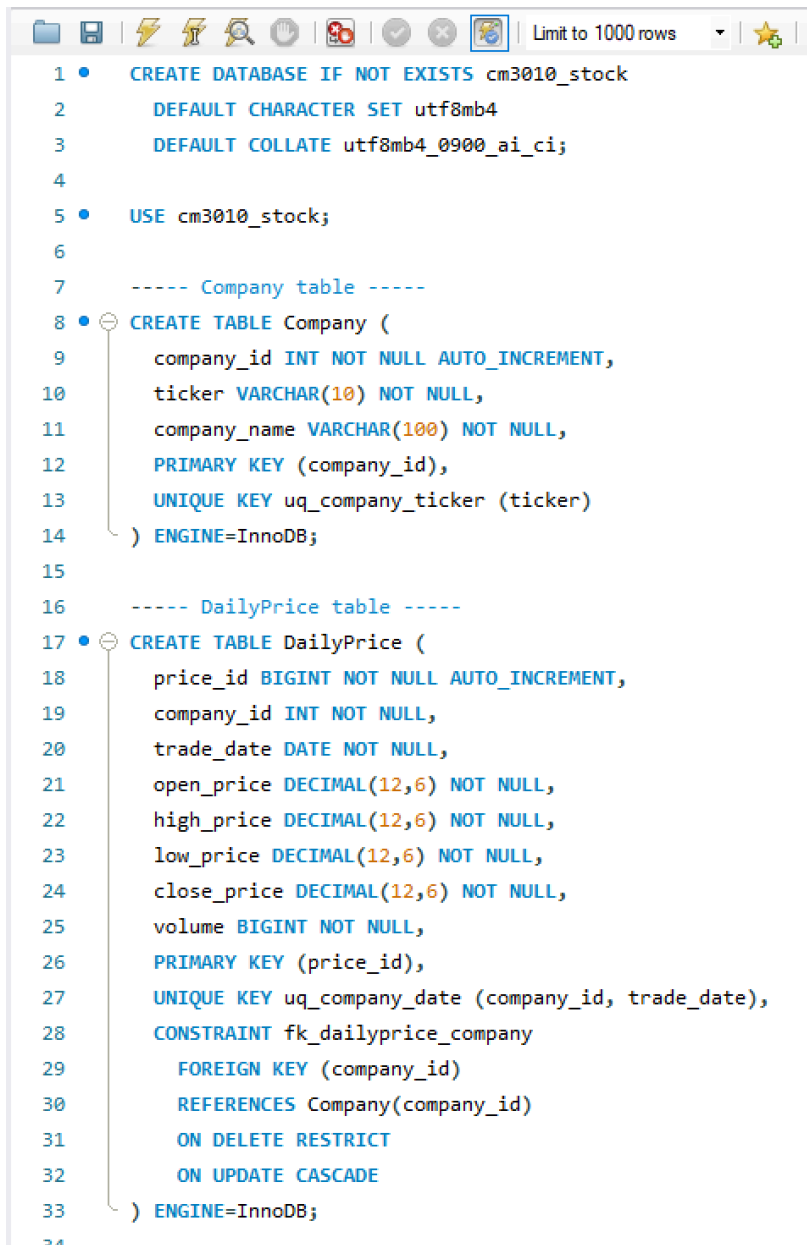
2.1 ER Diagram

The diagram consists of two main entities, Company and DailyPrice. Each company can have many daily price records, forming a one-to-many relationship.



2.2 Schema

- Company
 - company_id (primary key)
 - ticker
 - company_name
- DailyPrice
 - price_id (primary key)
 - company_id (foreign key -> company)
 - trade_date
 - open_price
 - high_price
 - low_price
 - close_price
 - volume



```
1 • CREATE DATABASE IF NOT EXISTS cm3010_stock
2     DEFAULT CHARACTER SET utf8mb4
3     DEFAULT COLLATE utf8mb4_0900_ai_ci;
4
5 • USE cm3010_stock;
6
7     ----- Company table -----
8 • CREATE TABLE Company (
9     company_id INT NOT NULL AUTO_INCREMENT,
10    ticker VARCHAR(10) NOT NULL,
11    company_name VARCHAR(100) NOT NULL,
12    PRIMARY KEY (company_id),
13    UNIQUE KEY uq_company_ticker (ticker)
14 ) ENGINE=InnoDB;
15
16     ----- DailyPrice table -----
17 • CREATE TABLE DailyPrice (
18    price_id BIGINT NOT NULL AUTO_INCREMENT,
19    company_id INT NOT NULL,
20    trade_date DATE NOT NULL,
21    open_price DECIMAL(12,6) NOT NULL,
22    high_price DECIMAL(12,6) NOT NULL,
23    low_price DECIMAL(12,6) NOT NULL,
24    close_price DECIMAL(12,6) NOT NULL,
25    volume BIGINT NOT NULL,
26    PRIMARY KEY (price_id),
27    UNIQUE KEY uq_company_date (company_id, trade_date),
28    CONSTRAINT fk_dailyprice_company
29        FOREIGN KEY (company_id)
30        REFERENCES Company(company_id)
31        ON DELETE RESTRICT
32        ON UPDATE CASCADE
33 ) ENGINE=InnoDB;
```

The screenshot shows a database IDE with the following SQL code in the editor:

```

27     UNIQUE KEY uq_company_date (company_id, trade_date),
28     CONSTRAINT fk_dailyprice_company
29     FOREIGN KEY (company_id)
30     REFERENCES Company(company_id)
31     ON DELETE RESTRICT
32     ON UPDATE CASCADE
33 ) ENGINE=InnoDB;
34
35 • DESCRIBE DailyPrice;

```

Below the code, the 'Result Grid' shows the table structure for 'DailyPrice':

Field	Type	Null	Key	Default	Extra
price_id	bigint	NO	PRI	NULL	auto_increment
company_id	int	NO	MUL	NULL	
trade_date	date	NO	MUL	NULL	
open_price	decimal(12,6)	NO		NULL	
high_price	decimal(12,6)	NO		NULL	
low_price	decimal(12,6)	NO		NULL	
close_price	decimal(12,6)	NO		NULL	
volume	bigint	NO		NULL	

The 'Output' pane shows the 'Action Output' for the 'DESCRIBE DailyPrice' command:

#	Time	Action
10	06:27:35	DESCRIBE DailyPrice

2.3 Design Decisions

A unique constraint was applied on the combination of (company_id, trade_date) to ensure that each company has at most one record per trading day. This prevents duplication of daily price data. The figure below shows the enforcement of this constraint, where an attempt to insert a second price record for the same company and date shows an error (error code 1062).

The screenshot shows a database IDE with the following SQL code in the editor:

```

49 • INSERT INTO DailyPrice
50 (company_id, trade_date, open_price, high_price, low_price, close_price, volume)
51 VALUES
52 (1, '2024-01-02', 100, 110, 95, 105, 1000000);
53

```

The 'Output' pane shows the 'Action Output' for the 'INSERT INTO DailyPrice' command:

#	Time	Action	Message	Duration / Fetch
19	06:28:59	DESCRIBE DailyPrice	8 row(s) returned	0.000 sec / 0.000 sec
20	06:29:05	DESCRIBE DailyPrice	8 row(s) returned	0.000 sec / 0.000 sec
21	06:35:17	INSERT INTO DailyPrice (company_id, trade_date, open_price, high_price, low_price, close_price, volume) VA	Error Code: 1062. Duplicate entry '1-2024-01-02' for key 'dailyprice.uq_company_date'	0.015 sec

I used surrogate primary keys for both tables. I enforced referential integrity using foreign key constraints. I separated the company metadata from time-series price data to avoid redundancy. I index on company_id and trade date to have efficient querying.

3.0 Database Implementation & Queries

3.1 Database implementation

The database was implemented using MySQL. All tables were created according to the schema. A dedicated database user with read-only (SELECT) privileges was created for use by the web application.

3.2 Data loading and Queries

Data was imported from CSV files into a staging table then inserted into the main DailyPrice table. Company rows were inserted first to generate company_id values. The staging table held the raw fields (trade_date, open, high, low, close, volume). For each ticker, rows were inserted into DailyPrice by selecting from the staging table and assigning the appropriate company_id. A unique constraint on company_id, trade_date prevented duplicate records. Row counts were validated per ticker after import.

This query shows the summary of data for each company. (record count & date range)

```
1  -- Dataset summary: record count and date range per ticker
2  • SELECT
3      c.ticker,
4      COUNT(*) AS row_count,
5      MIN(d.trade_date) AS min_date,
6      MAX(d.trade_date) AS max_date
7  FROM DailyPrice d
8  JOIN Company c
9      ON c.company_id = d.company_id
10 GROUP BY c.ticker
11 ORDER BY c.ticker;
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
ticker	row_count	min_date	max_date
AAPL	1748	2019-01-02	2025-12-12
AMZN	1748	2019-01-02	2025-12-12
GOOGL	1748	2019-01-02	2025-12-12
META	1748	2019-01-02	2025-12-12
MSFT	1748	2019-01-02	2025-12-12
NVDA	1748	2019-01-02	2025-12-12
TSLA	1748	2019-01-02	2025-12-12

This query fetches the overall return values

```

1  -- Overall percentage return per company
2  WITH bounds AS (
3      SELECT
4          company_id,
5          MIN(trade_date) AS first_date,
6          MAX(trade_date) AS last_date
7      FROM DailyPrice
8      GROUP BY company_id
9  )
10 SELECT
11     c.ticker,
12     ROUND((MAX(d.close_price) / MIN(d.close_price) - 1) * 100, 2) AS pct_return
13 FROM DailyPrice d
14 JOIN bounds b
15     ON d.company_id = b.company_id
16 JOIN Company c
17     ON c.company_id = d.company_id
18 GROUP BY c.ticker
19 ORDER BY pct_return DESC;
20

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
ticker	pct_return		
NVDA	6423.27		
TSLA	3921.86		
META	788.54		
AAPL	743.84		
GOOGL	533.10		
MSFT	490.46		
AMZN	238.60		

This query fetches the volatility values

```

1  -- Daily return volatility per company
2  WITH returns AS (
3      SELECT
4          company_id,
5          close_price / LAG(close_price)
6          OVER (PARTITION BY company_id ORDER BY trade_date) - 1 AS daily_return
7      FROM DailyPrice
8  )
9  SELECT
10     c.ticker,
11     ROUND(STDDEV_SAMP(daily_return) * 100, 4) AS volatility
12 FROM returns r
13 JOIN Company c
14     ON c.company_id = r.company_id
15 WHERE daily_return IS NOT NULL
16 GROUP BY c.ticker
17 ORDER BY volatility DESC;
18

```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
ticker	volatility		
TSLA	4.0607		
NVDA	3.2551		
META	2.6467		
AMZN	2.1578		
GOOGL	1.9801		
AAPL	1.9631		
MSFT	1.7924		

This is the query for the explore page, where users can choose parameters for the company and number of days

```
1  -- Explore: recent price history for a selected ticker
2  -- Parameters:
3  -- :ticker → selected stock ticker
4  -- :days  → number of recent trading days
5
6  • SELECT
7      d.trade_date,
8      d.open_price,
9      d.high_price,
10     d.low_price,
11     d.close_price,
12     d.volume
13 FROM DailyPrice d
14 JOIN Company c
15     ON c.company_id = d.company_id
16 WHERE c.ticker = 'AAPL'
17 ORDER BY d.trade_date DESC
18 LIMIT 10;
```

	trade_date	open_price	high_price	low_price	close_price	volume
►	2025-12-12	277.900000	279.220000	276.820000	278.280000	39532887
	2025-12-11	279.095000	279.590000	273.810000	278.030000	33247986
	2025-12-10	277.750000	279.750000	276.440000	278.780000	33038318
	2025-12-09	278.160000	280.030000	276.920000	277.180000	32193256
	2025-12-08	278.130000	279.669300	276.150000	277.890000	38211832
	2025-12-05	280.540000	281.140000	278.050000	278.780000	47265845
	2025-12-04	284.095000	284.730000	278.590000	280.700000	43989056
	2025-12-03	286.200000	288.620000	283.300000	284.150000	43538687
	2025-12-02	283.000000	287.400000	282.630100	286.190000	53669532
	2025-12-01	278.010000	283.420000	276.140000	283.100000	46587722

These queries are directly used by the Node.js application to populate each analytical view

4.0 Web Application

A simple web application was developed using Node.js and Express to provide interactive access to the stock database.

4.1 Architecture

The application connected to the MySQL database using the mysql2 library. Key features include use of environment variables for configuration, parameterised queries for user-selected inputs, and a dedicated read-only database user to enforce appropriate privileges.

Functions include:

Summary page, to display record counts and date ranges per company

Returns page, to rank companies by overall percentage return

Volatility page, to show daily return volatility per company

Average page, to compare average closing prices across companies.

CM3010 Stage 4 Web App

Interactive Node.js + MySQL application for stock analysis.

- Dataset coverage
- Returns
- Volatility
- Volume
- User-driven exploration

Summary

ticker	Rows	min_date	max_date
AAPL	1,748	Wed Jan 02 2019 00:00:00 GMT+0800 (Singapore Standard Time)	Fri Dec 12 2025 00:00:00 GMT+0800 (Singapore Standard Time)
AMZN	1,748	Wed Jan 02 2019 00:00:00 GMT+0800 (Singapore Standard Time)	Fri Dec 12 2025 00:00:00 GMT+0800 (Singapore Standard Time)
GOOGL	1,748	Wed Jan 02 2019 00:00:00 GMT+0800 (Singapore Standard Time)	Fri Dec 12 2025 00:00:00 GMT+0800 (Singapore Standard Time)
META	1,748	Wed Jan 02 2019 00:00:00 GMT+0800 (Singapore Standard Time)	Fri Dec 12 2025 00:00:00 GMT+0800 (Singapore Standard Time)
MSFT	1,748	Wed Jan 02 2019 00:00:00 GMT+0800 (Singapore Standard Time)	Fri Dec 12 2025 00:00:00 GMT+0800 (Singapore Standard Time)
NVDA	1,748	Wed Jan 02 2019 00:00:00 GMT+0800 (Singapore Standard Time)	Fri Dec 12 2025 00:00:00 GMT+0800 (Singapore Standard Time)
TSLA	1,748	Wed Jan 02 2019 00:00:00 GMT+0800 (Singapore Standard Time)	Fri Dec 12 2025 00:00:00 GMT+0800 (Singapore Standard Time)

Returns

ticker	pct_return
NVDA	6423.27
TSLA	3921.86
META	788.54
AAPL	743.84
GOOGL	533.10
MSFT	490.46
AMZN	238.60

Volatility

ticker	volatility
TSLA	4.0607
NVDA	3.2551
META	2.6467
AMZN	2.1578
GOOGL	1.9801
AAPL	1.9631
MSFT	1.7924

Volume

ticker	avg_volume
NVDA	418994641
TSLA	123699434
AAPL	90713872
AMZN	66606471
GOOGL	33295976
MSFT	27878014
META	21119129

Average Close

ticker	company_name	avg_close
META	Meta	334.35
MSFT	Microsoft	289.03
TSLA	Tesla	204.03
AAPL	Apple	148.08
AMZN	Amazon	148.05
GOOGL	Alphabet	122.01
NVDA	NVIDIA	49.05

4.2 User-Driven Querying

An Explore page was implemented to allow user-controlled querying of the dataset. Users can select a stock ticker, choose the number of trading days using a slider, and switch between price history and summary statistics. User inputs are validated before being passed into SQL queries, ensuring safe and meaningful interactions.

CM3010 - Stock Dashboard

HomeSummaryReturnsVolatilityVolumeAverageExplore

Explore

AAPLPrices

Run

	trade_date	open_price	high_price	low_price	close_price	volume
AAPL						
AMZN						
GOOGL	2 2025 00:00:00 GMT+0800 (Singapore Standard Time)	277.900000	279.220000	276.820000	278.280000	39,532,887
META	11 2025 00:00:00 GMT+0800 (Singapore Standard Time)	279.095000	279.590000	273.810000	278.030000	33,247,986
MSFT						
NVDA	10 2025 00:00:00 GMT+0800 (Singapore Standard Time)	277.750000	279.750000	276.440000	278.780000	33,038,318
TSLA						
	Tue Dec 09 2025 00:00:00 GMT+0800 (Singapore Standard Time)	278.160000	280.030000	276.920000	277.180000	32,193,256
	Mon Dec 08 2025 00:00:00 GMT+0800 (Singapore Standard Time)	278.130000	279.669300	276.150000	277.890000	38,211,832
	Fri Dec 05 2025 00:00:00 GMT+0800 (Singapore Standard Time)	280.540000	281.140000	278.050000	278.780000	47,265,845
	Thu Dec 04 2025 00:00:00 GMT+0800 (Singapore Standard Time)	284.095000	284.730000	278.590000	280.700000	43,989,056
	Wed Dec 03 2025 00:00:00 GMT+0800 (Singapore Standard Time)	286.200000	288.620000	283.300000	284.150000	43,538,687
	Tue Dec 02 2025 00:00:00 GMT+0800 (Singapore Standard Time)	283.000000	287.400000	282.630100	286.190000	53,669,532
	Mon Dec 01 2025 00:00:00 GMT+0800 (Singapore Standard Time)	278.010000	283.420000	276.140000	283.100000	46,587,722
	Fri Nov 28 2025 00:00:00 GMT+0800 (Singapore Standard Time)	277.260000	279.000000	275.986500	278.850000	20,135,620
	Wed Nov 26 2025 00:00:00 GMT+0800 (Singapore Standard Time)	276.960000	279.530000	276.630000	277.550000	33,431,423
	Tue Nov 25 2025 00:00:00 GMT+0800 (Singapore Standard Time)	275.270000	280.380000	275.250000	276.970000	46,914,220
	Mon Nov 24 2025 00:00:00 GMT+0800 (Singapore Standard Time)	270.900000	277.000000	270.900000	275.920000	65,585,796
	Fri Nov 21 2025 00:00:00 GMT+0800 (Singapore Standard Time)	265.950000	273.330000	265.670000	271.490000	59,030,832

CM3010 - Stock Dashboard

HomeSummaryReturnsVolatilityVolumeAverageExplore

Explore

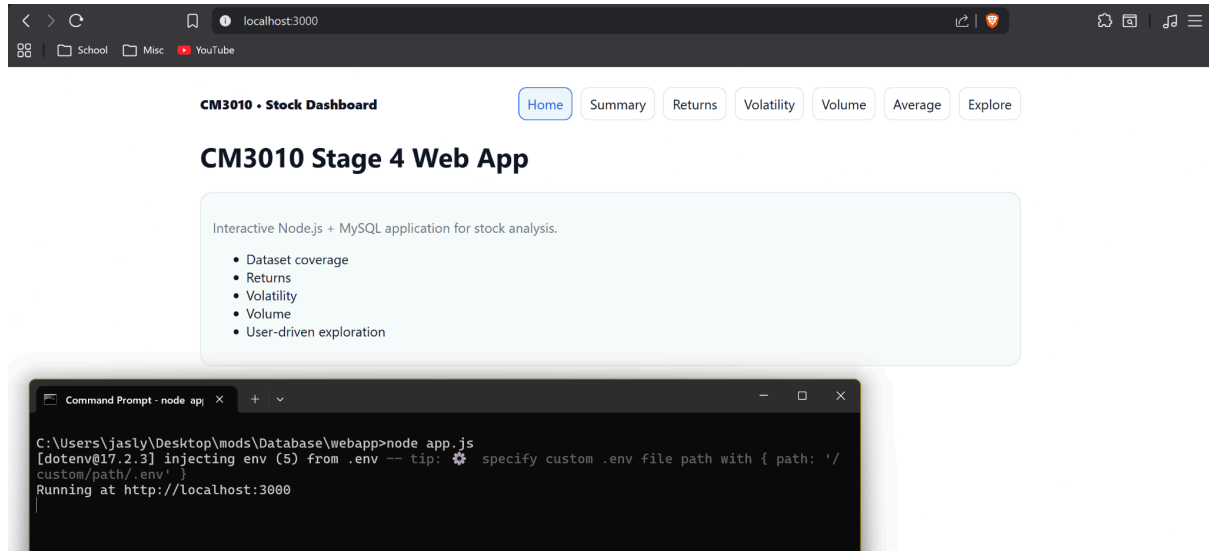
GOOGLStats

Run

ticker	min_close	max_close	avg_close	avg_volume
GOOGL	147.67	323.44	218.22	38179224

4.3 Execution Evidence

The application was successfully executed using Node.js, confirming that the web application runs and interacts correctly with the database.



5.0 Conclusion

This project shows the entire process from designing a database and building a data model to interactively exploring data. Using relational database design, SQL querying and Node.js, insights into stock price behaviour was successfully extracted and shown in a meaningful manner.